VGPP

Modular Monolith Design

Objective:

1. Keep the application as one deployable unit (monolith) for simplicity.
2. Maintain clear modular boundaries so that it's easily scalable in future.
3. Ensure data ownership, separations of concerns for future extraction into microservices

Define Modules Based on Business Domains:

Module	Responsibility	Key Features
User Module	Manage bank manager and admin accounts	Registration, login, authentication, profile management
Catalog Module	Manage products	Product list, product id, product name, product details, pricing
Order Module	Handle order lifecycle	Create order, manage cart, calculate totals, order history
Payment Module	Payment processing	Capture payments, track status, store reference IDs, order confirmation, payment confirmation
Shipment/Dispatch (part of Order)	Track shipment via LR	Store LR number, transporter, tracking URL
Notification (Optional)	Email notifications	Registration, LR email

Define Module Communication:

Internal module calls: Use service interfaces (no direct repository access across modules)
Data flow example:

1. Order module calls Payment module service to process payments
2. Shipment info is updated in Order module after dispatch

Rules:

1. Modules own their data
2. No shared tables
3. Interaction only through well-defined interfaces

Project Structure:

```
vgpp-app/
 ├── user/
 │    ├── controller/     # UserController
 │    ├── service/        # UserService
 │    ├── repository/     # UserRepository
 │    └── model/          # UserEntity
 │
 ├── catalog/
 │    ├── controller/     # ProductController
 │    ├── service/        # ProductService
 │    ├── repository/     # ProductRepository
 │    └── model/          # ProductEntity
 │
 ├── order/
 │    ├── controller/     # OrderController
 │    ├── service/        # OrderService
 │    ├── repository/     # OrderRepository
 │    └── model/          # OrderEntity, ShipmentInfo
 │
 ├── payment/
 │    ├── controller/     # PaymentController
 │    ├── service/        # PaymentService
 │    ├── repository/     # PaymentRepository
 │    └── model/          # PaymentEntity
 │
 └── common/
      ├── config/         # Security, JWT, etc.
      ├── exceptions/     # Custom exceptions
      └── utils/          # Shared helpers
```

This structure mirrors microservice boundaries internally. Keeps modules independent, so extracting them later into separate Spring Boot services is easy.

Database Design

Each module owns its **own database tables**:

1. Users: users
2. Catalog: products
3. Order: orders, order_items, order_shipment
4. Payment: payment

No cross-module table access.
Relationships handled via IDs (foreign key reference in service logic)

Data Flow Example:

1. Bank manager logs in via User Module
2. Adds products from catalog module to order module
3. Proceeds to payment through payment module
4. After payment success assigns LR number to respective Order module
5. Sends optional notifications

But each module should handle its own responsibilities, communication only via service interfaces, Modules are loosely coupled so that extraction to microservices can be easy.

Why this works for VGPP:

1. Low traffic
2. Single team
3. Future proof